

Here is a complete, production-ready Command Line Interface (CLI) weather application written in Python. It uses the requests library to fetch real-time data from the free WeatherAPI service.

Prerequisites

You will need to install the requests library if you haven't already:
pip install requests

Note: To run this, you'll need a free API key from [WeatherAPI.com](https://weatherapi.com). Replace the placeholder YOUR_API_KEY in the code with your actual key.

The Code

```
import sys
import requests

def get_weather(city: str, api_key: str):
    """
    Fetches real-time weather data for a given city using WeatherAPI.
    """
    # Base URL for WeatherAPI current weather endpoint
    base_url = "http://api.weatherapi.com/v1/current.json"

    # HTTP query parameters required by the API
    params = {
        "key": api_key,
        "q": city,
        "aqi": "no" # We don't need Air Quality Index for this basic app
    }

    try:
        # Send a GET request to the API
        response = requests.get(base_url, params=params)

        # Raise an exception if the HTTP request returned an error code (e.g., 404, 401)
        response.raise_for_status()

        # Parse the JSON response into a Python dictionary
        data = response.json()
        return data

    except requests.exceptions.HTTPError:
        # Specific handling for HTTP errors (like an invalid API key or city not found)
        if response.status_code == 400:
            print(f"❌ Error: City '{city}' not found. Please check the spelling.")
        elif response.status_code == 403:
            print(f"❌ Error: Invalid API Key. Please check your credentials.")
        else:
```

```
    print(f"❌ HTTP Error occurred: {response.status_code}")
    return None
```

```
except requests.exceptions.ConnectionError:
    print(f"❌ Network Error: Could not connect to the weather service. Check your internet.")
    return None
```

```
except requests.exceptions.Timeout:
    print(f"❌ Timeout Error: The request took too long. Try again later.")
    return None
```

```
except Exception as e:
    print(f"❌ An unexpected error occurred: {e}")
    return None
```

```
def display_weather(data: dict):
    """
    Parses and beautifully prints the weather data.
    """
    if not data:
        return

    # Extracting specific data points from the JSON structure
    location = data["location"]["name"]
    region = data["location"]["region"]
    country = data["location"]["country"]

    temp_c = data["current"]["temp_c"]
    condition = data["current"]["condition"]["text"]
    humidity = data["current"]["humidity"]
    wind_kph = data["current"]["wind_kph"]
    feelslike_c = data["current"]["feelslike_c"]

    # Formatting the CLI Output
    print("\n" + "="*40)
    print(f"🌍 Weather for: {location}, {region}, {country}")
    print("="*40)
    print(f"🌡️ Temperature: {temp_c}°C (Feels like: {feelslike_c}°C)")
    print(f"☁️ Condition: {condition}")
    print(f"💧 Humidity: {humidity}%")
    print(f"💨 Wind Speed: {wind_kph} km/h")
    print("="*40 + "\n")
```

```
def main():
    # Replace this with your actual WeatherAPI key
    API_KEY = "YOUR_API_KEY"

    if API_KEY == "YOUR_API_KEY":
```

```

    print("⚠️ Configuration Error: Please update the API_KEY variable with your WeatherAPI
token.")
    sys.exit(1)

print("☀️ Welcome to the CLI Weather App! ☀️")

# Keep the CLI app running in a loop until the user decides to exit
while True:
    try:
        city = input("Enter a city name (or type 'exit' to quit): ").strip()

        # Check for exit condition
        if city.lower() == 'exit':
            print("Goodbye! Have a great day!")
            break

        # Skip empty inputs
        if not city:
            print("Please enter a valid city name.")
            continue

        # Fetch and display the data
        weather_data = get_weather(city, API_KEY)
        display_weather(weather_data)

    except KeyboardInterrupt:
        # Clean exit handling if the user presses Ctrl+C
        print("\nGoodbye!")
        break

if __name__ == "__main__":
    main()

```

Key Features Implemented:

- **Robust Error Handling:** It intercepts 404/400 errors (invalid city names), 403 errors (bad API keys), and network timeouts/disconnects without crashing.
- **Persistent Loop:** The CLI stays active so you can check multiple cities back-to-back without restarting the script.
- **Clean Layout:** Uses structured spacing and clear ASCII/Emoji UI elements to make the terminal data easy to read.